SOFTWARE DESIGN PATTERNS / ASSIGNMENT 5

SMART PARKING GARAGE

STRATEGY PATTERN • UML • UNIT TESTS • GIT REFACTORING

Technical project report and defense notes

Student: Ali Ilyubayev | Group: SE-2532 | Year: 2026

Implementation: Java 17+, Maven, JUnit 5. Domain: multi-level parking garage. Example currency: KZT.

Project abstract

This project demonstrates runtime replacement of algorithms in two independent operations: pricing a parking stay and choosing a suitable free garage space. Six concrete strategies implement two small interfaces. Two contexts delegate without type-based branching, while separate selectors implement automatic business rules. A simulated occupancy sensor is hidden behind an interface; a fake source enables deterministic tests. The project includes a runnable demonstration, UML, and preserved Git refactoring history.

Quick facts

Area	Delivered implementation
Pricing strategies	Hourly; daily cap; occupancy-driven demand
Allocation strategies	Nearest entrance; lowest level; least crowded level
Automatic selection	PricingSelector; AllocationSelector
Runtime swapping	PricingContext.setStrategy(); AllocationContext.setStrategy()
Quality evidence	30 JUnit test methods; 32 independently checked invocations; 3 preserved regression checks
Git history	before-strategy and after-strategy tags; unchanged regression test source

Source: Assignment 5, “Strategy Pattern: Interchangeable Algorithms, Runtime Selection, Clean Classes.” Business prices and garage data are illustrative.

1. Assignment requirements and implementation

Requirement from assignment	Where it is satisfied
Two families; interface with 1–2 methods; 3 distinct classes each	PricingStrategy with HourlyPricing / DailyCapPricing / DemandPricing; SpotAllocationStrategy with NearestEntrance / LowestLevel / LeastCrowdedLevel
One context per family; constructor; setStrategy; pure delegation	PricingContext and AllocationContext; no branch on concrete types
Separate automatic selector	PricingSelector and AllocationSelector; explicit prioritized rules
Refactor bad pricing implementation with Git; preserve tests	a01eb84 before; 1c04bcf after; PricingBehaviorRegressionTest unchanged
Isolate sensor data; test with fake	OccupancySource; SimulatedOccupancySource; FakeOccupancySource
JUnit 5, each strategy, selector rules, parameterized check	Six JUnit test classes; 32 checks in offline harness
Main demo with identical inputs, manual/auto choice, runtime change	SmartParkingDemo.main()
README: SRP table, comparison, links	README.md; docs/DEFENSE_GUIDE.md; docs/COMMITS.md

2. Architecture overview

The architecture divides the garage into shared request/space data, a sensor boundary, a pricing strategy family, and a space-allocation strategy family. Every concrete strategy implements one interface. Contexts depend on interfaces rather than concrete algorithms. Each selector owns policy rules; it returns a strategy object to the caller.

- Interface = shared method contract; concrete strategy = independent algorithm implementation.
- Context = holds the active strategy, forwards requests, replaces strategy through setStrategy().
- Selector = chooses a strategy from membership, stay duration, occupancy, or accessibility.
- OccupancySource = abstraction separating pricing from an external sensor implementation.

The full editable UML source is docs/uml/smart-parking-garage.puml. The project ZIP includes a high-resolution full-project PNG plus larger diagrams for both families.

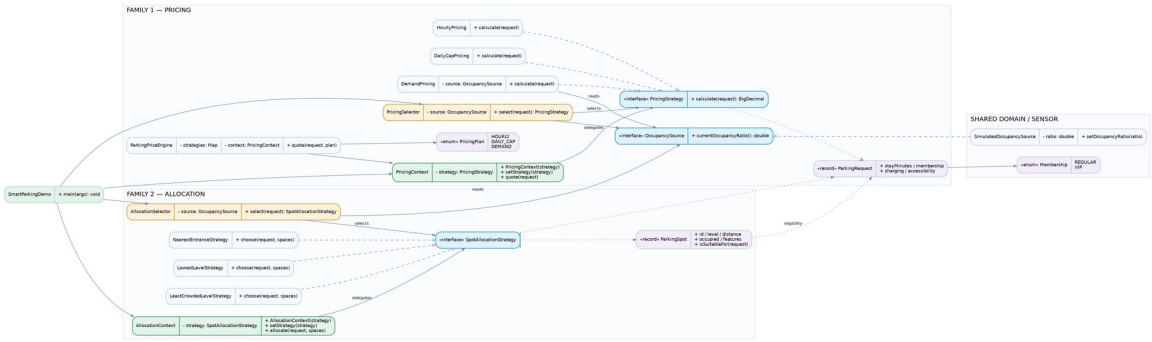


Figure 1. Full class UML overview. Open the included PNG for detail.

3. Pricing Strategy family

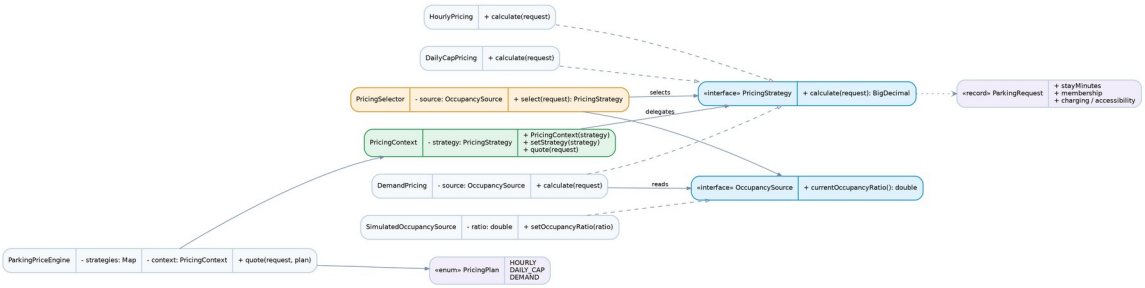


Figure 2. Pricing UML with interface implementations, context, selector, sensor boundary.

Concrete pricing algorithm	Formula / step	Example
HourlyPricing	ceil(minutes / 60) × 500	61 min = 1,000 KZT
DailyCapPricing	For each 24h block: min(ceil(blockMinutes / 60) × 500, 3,000)	26 h = 3,000 + 1,000 = 4,000 KZT
DemandPricing	(600 + ceil(minutes / 30) × 250) × occupancy multiplier	60 min, 90% used = 1,650 KZT

Demand occupancy multiplier: below 70% = 1.0; 70% up to below 85% = 1.2; at least 85% = 1.5. Money amounts are represented with BigDecimal.

PricingSelector decision order

- VIP plus stay ≥ 180 minutes → DailyCapPricing.
- Otherwise occupancy ≥ 85% → DemandPricing.
- Otherwise stay ≥ 360 minutes → DailyCapPricing.
- Otherwise → HourlyPricing.

This priority order ensures that eligible VIP long stays retain the cap even under high congestion. The pricing context has no conditional choice logic; the selector owns the choice.

4. Parking-space allocation family

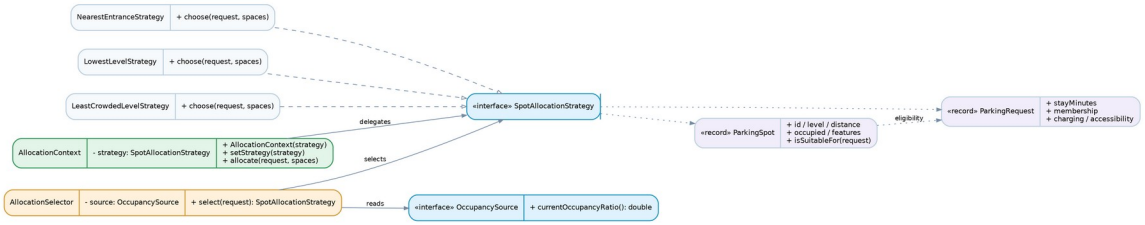


Figure 3. Allocation UML with the second interface, strategies, context, selector.

Concrete allocation algorithm	Decision criterion
NearestEntranceStrategy	Shortest distance among currently free, feature-compatible spaces.
LowestLevelStrategy	Smallest level number, then shortest distance, then spot ID.
LeastCrowdedLevelStrategy	Highest free-space fraction on a floor, then level, distance, spot ID.

Every allocation algorithm first excludes occupied spaces. Electric charging is required when electricCharging=true; accessibility is required when accessibleSpace=true. When no suitable free space exists, the result is Optional.empty(), not a fictitious reservation.

AllocationSelector decision order

- Accessible parking requested → NearestEntranceStrategy.
- Otherwise occupancy ≥ 80% → LeastCrowdedLevelStrategy.
- Otherwise VIP → NearestEntranceStrategy.
- Otherwise → LowestLevelStrategy.

Concrete example: floor 0 is heavily occupied while floor 2 is empty. LowestLevelStrategy still prefers floor 0; LeastCrowdedLevelStrategy spreads arrivals toward floor 2. This difference illustrates truly distinct algorithms.

5. Production class responsibilities

One-sentence responsibilities are also provided in the README; each respects the assignment's word limit and wording rule.

Class / interface	One responsibility
Membership	Represents supported membership tiers.
ParkingRequest	Holds immutable inputs for one customer stay.
ParkingSpot	Represents one garage space with suitability metadata.
OccupancySource	Defines the occupancy-reading contract.
SimulatedOccupancySource	Supplies adjustable simulated occupancy readings.
PricingPlan	Names supported manual pricing plans.
PricingStrategy	Defines the interchangeable price-calculation contract.
HourlyPricing	Computes fees from started hours.
DailyCapPricing	Computes a capped price per daily block.
DemandPricing	Computes occupancy-sensitive prices from half-hour units.
PricingContext	Delegates fee calculation to its active strategy.
PricingSelector	Chooses a pricing policy from current conditions.
ParkingPriceEngine	Translates manual plan choices into strategy objects.
SpotAllocationStrategy	Defines the interchangeable space-selection contract.
NearestEntranceStrategy	Finds the suitable space with minimal walking distance.
LowestLevelStrategy	Finds a suitable space on the lowest garage level.
LeastCrowdedLevelStrategy	Finds a suitable space on the floor with most spare capacity.
AllocationContext	Delegates space choice to its active strategy.
AllocationSelector	Chooses a space-selection policy from current conditions.
SmartParkingDemo	Demonstrates both strategy families using a console scenario.

6. Unit testing and change isolation

JUnit 5 class	What it verifies
PricingBehaviorRegressionTest	The same three pricing outcomes before and after refactoring.
PricingStrategiesTest	Individual pricing algorithms, demand tiers, shared nonnegative rule via @ParameterizedTest.
PricingSelectorTest	Every automatic pricing rule, priority, boundary.
AllocationStrategiesTest	Every space algorithm, equipment filters, unavailable-space behavior.
AllocationSelectorTest	Every automatic allocation rule, including conflicting priority scenarios.
ContextsAndSourcesTest	Runtime switching, source replacement, input validation.
FakeOccupancySource	Stable readings, allowing tests without connecting to a sensor.

The project contains 30 JUnit test methods. One parameterized method yields three cases, producing 32 test invocations. The authoring environment did not have Maven/JUnit binaries or a reachable dependency server. We compiled the production application with javac, ran the main demo, then verified all test assertion paths using a temporary offline Java assertion/annotation harness: 32 checks, 0 failures. Actual mvn test must still be run on a machine with Maven dependencies.

Before/after regression

The before-strategy tag contains the original ParkingPriceEngine with a large if/else block. The after-strategy tag extracts each pricing algorithm into its own class and makes the stable engine delegate through PricingContext. The three original regression tests ran successfully on both snapshots without changing the original test source.

Git snapshot	Revision prefix	Verified assertion checks
BEFORE	a01eb84	3 passed / 0 failed
AFTER	1c04bcf	32 passed / 0 failed

7. Why this is Strategy, not State or Template Method

Strategy is appropriate because the parking garage must switch among independent algorithms for a single operation. Every pricing implementation receives the same request and returns a BigDecimal, while every space-selection implementation receives the same request and list of spaces, returning an Optional<ParkingSpot>. Each family has a stable interface. The context stores an object implementing that interface, delegates work to it, and permits explicit runtime replacement. A caller can choose an algorithm manually or let a selector decide. This reduces coupling and keeps new algorithm implementations separate from the context.

State is different: State pattern objects normally model lifecycle conditions of an owner, often with transitions as the owner changes behavior. The parking strategies represent business policies rather than stages in a garage lifecycle. An individual strategy does not cause the context to transition to another internal stage. The caller or selector chooses the next policy.

Template Method is different: it defines a fixed sequence of steps in a base class, leaving selected steps for subclasses to override. Smart Parking Garage does not use such an inheritance hierarchy. Each strategy independently fulfills an interface; the shared behavior is delegation through composition, not a fixed inherited algorithm.

Open/Closed Principle: an extra concrete strategy can be written against the existing interface and injected into the current context without editing its implementation. A new automatic business condition may require a focused change inside the selector, the designated location for selection policy.

8. Live defense plan

- Run java edu.aitu.parking.app.SmartParkingDemo or mvn exec:java.
- Point out the three pricing outputs for the same 120-minute request plus the additional 9-hour example.
- Show the different space assignments for nearest, lowest level, least crowded level.
- Change simulated occupancy to 92%; describe automatic pricing and allocation strategy changes.
- Explain the two context constructors, setStrategy(), delegation, and the two selector methods.
- Show the before/after tags in git log; demonstrate the unchanged regression tests.
- Open the UML source; explain interface implementation arrows and context dependencies.

9. Representative defense questions

Teacher question	Concise answer
Why two families?	Price calculation and spot allocation have different contracts.
Where may if/switch select strategies?	Inside selector classes, never inside the contexts.
How does sensor isolation work?	DemandPricing depends on OccupancySource; tests inject FakeOccupancySource.
How is runtime swapping shown?	setStrategy() changes the implementation in the same context instance.
What about a full garage?	No eligible spot yields Optional.empty().
How can you add a new algorithm?	Implement the existing strategy interface; inject the new object.